
Restrict, contain, expand

Getting AI to do less, so you can trust it with more.

Your problems today, based on insights

It over-engineers

Ask for a fix and you get a framework: layers and options nobody asked for.

It touches too much

A small change arrives with edits to files that have nothing to do with it.

It uses too many tokens

Whole files and long sessions go into the context for a change of a few lines.

Main use is diagnostics, not a full solution

It explains what is wrong. The fix is still written by hand.

Where AI already shines is as a teacher.

A new technology or approach, learned in days rather than weeks. And a working familiarity with far more of the stack than any one person reaches alone.

Rules

Interrupt me

If you have a question, ask it when it comes up.

Question me

Challenge my suggestions and my workflow, and make it your own.

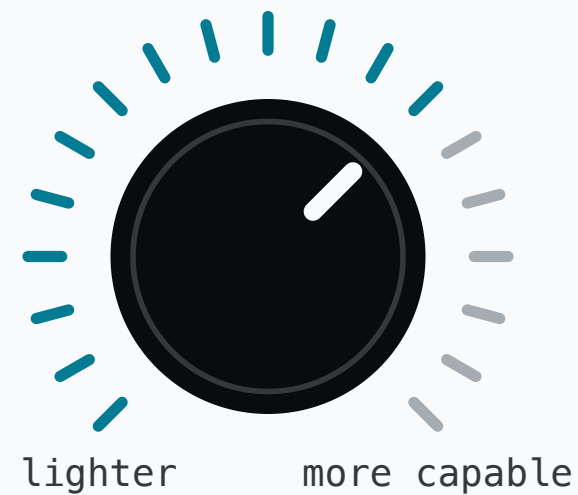
Experiment

Learn by doing, not by theory.

No secrets

I have no tricks to share. The simplest things are the ones that work.

The dials you control



Model

Buys Better reasoning on hard problems.

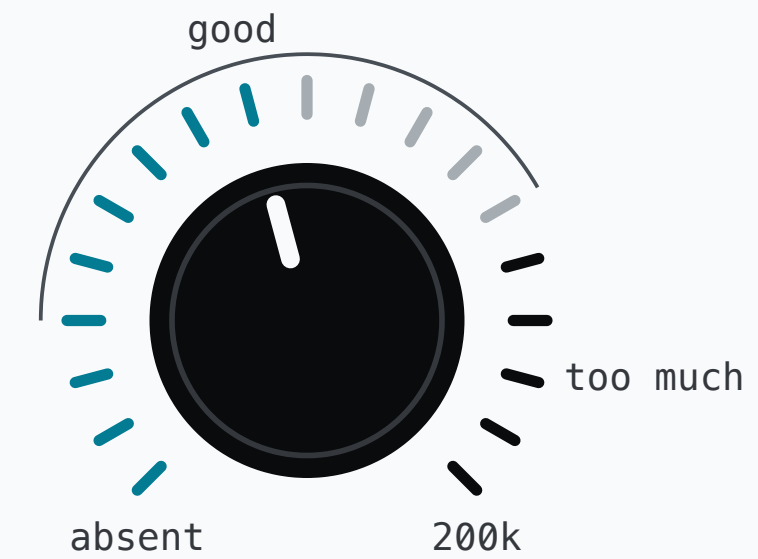
Costs A higher price on every token it reads and writes.



Effort

Buys More thinking before it answers.

Costs Slower answers, and the thinking is billed as tokens too.



Context

Buys What it needs to know to do the job.

Costs Every token in it, billed again on every turn.

Risks Wrong context leads it astray.

Where your context comes from

Always there

System prompt and tools Global MCP servers Global skills Global CLAUDE.md

For most people that is 30k to 50k tokens before they send anything. Run `/context` to see yours.

Per project

Project CLAUDE.md Project skills Project agents Project MCP servers

CLAUDE.md in subfolders

Checked into the repository, so everyone who works on it carries the same load.

Per task

Your prompt Resources you hand it Files it reads

The only layer you write fresh every time.

The prompt carries the intent

Prompt for the work

What most people write

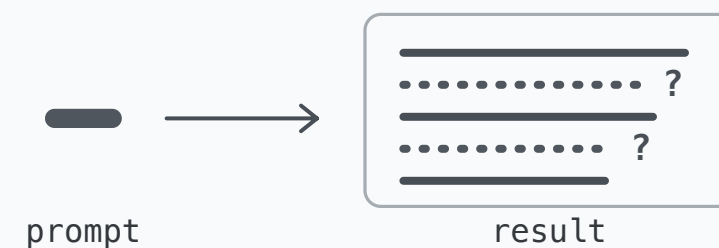
— ~100 tokens

The rest is guessed.

A line or two asking for the result

The gaps get filled with assumptions

More effort and a bigger model to make up for them



Prompt for the prompt

Let Claude interview you first

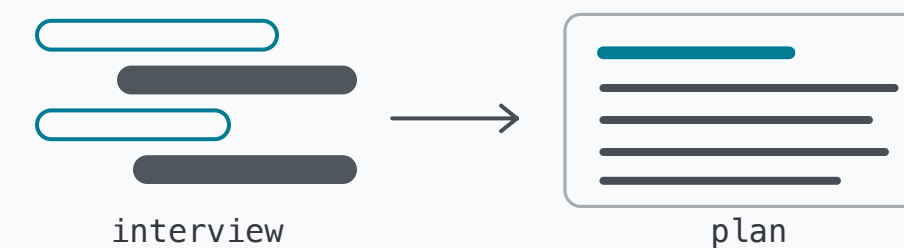
— ~1,500 tokens

High-quality tokens, and well spent.

A plan with the details and the whole intent in it

The interview raises what you had not thought of

Effort and model can come down for the long run



Where your context comes from

Always there

System prompt and tools Global MCP servers Global skills Global CLAUDE.md

For most people that is 30k to 50k tokens before they send anything. Run `/context` to see yours.

Per project

Project CLAUDE.md Project skills Project agents Project MCP servers

CLAUDE.md in subfolders

Checked into the repository, so everyone who works on it carries the same load.

Per task

Your prompt Resources you hand it Files it reads

The only layer you write fresh every time.

Most of the context is your code

The files it reads while working through a task are the biggest share of the context, and the cheapest lever you have. If those are bad, so is the result.

Name things well

Variables, functions and entrypoints that say what they are.

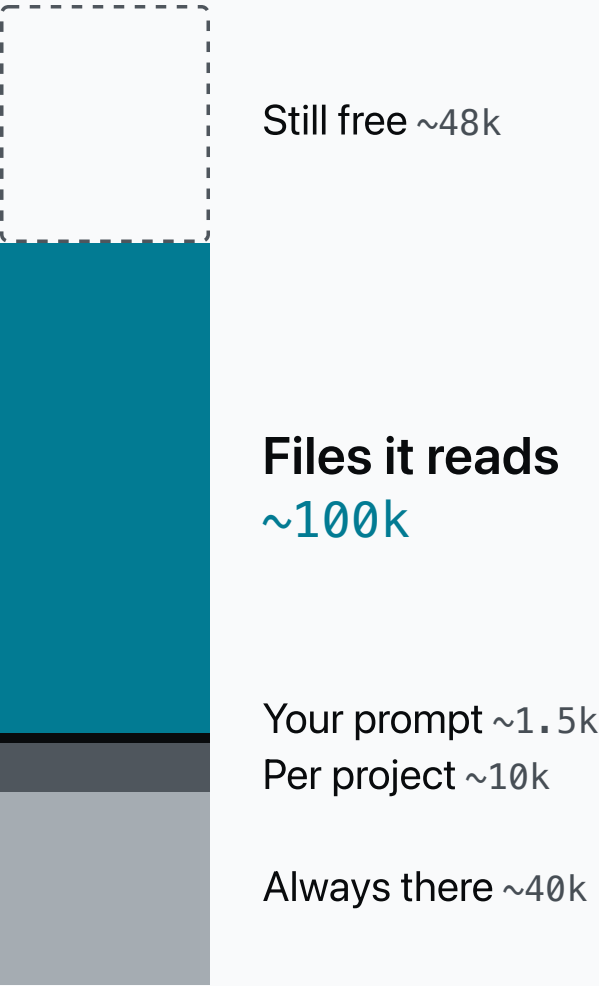
Keep one structure

If it confuses you, it will confuse the agent.

Follow the community

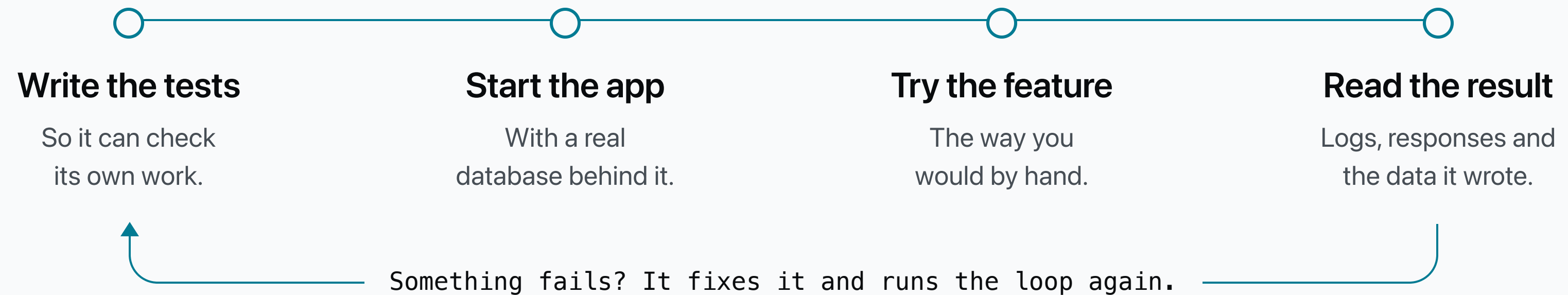
It already knows the standards, so they need no explaining.

Context window · 200k



Prove your work

You run and test a feature before you call it done. Give the agent the same tools you use, in an environment where it can do the same.



The best results come when the agent can work end to end, without waiting on you.

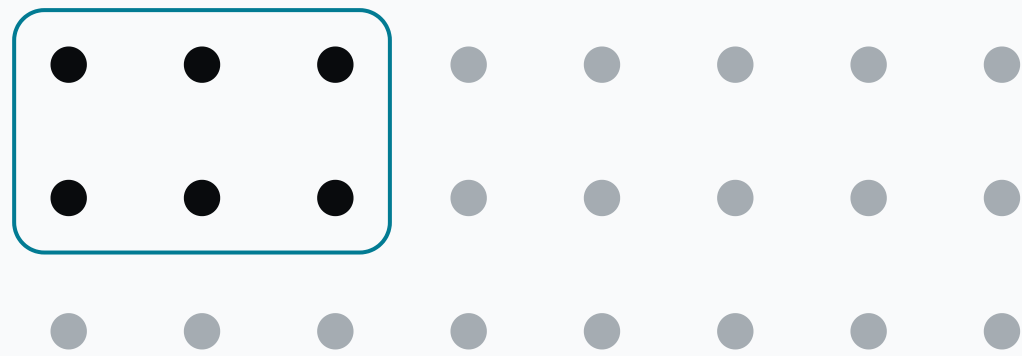
We are all for RCE

Not remote code execution. Restrict, contain, expand, in that order.

R

Restrict

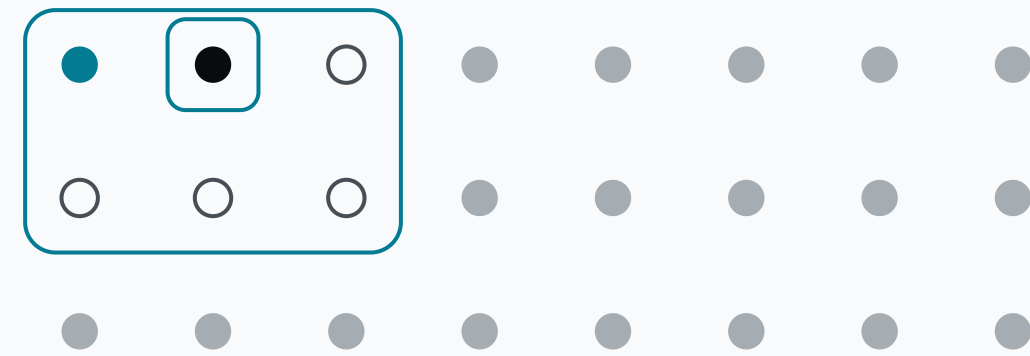
Stop the AI from doing more than it should.



C

Contain

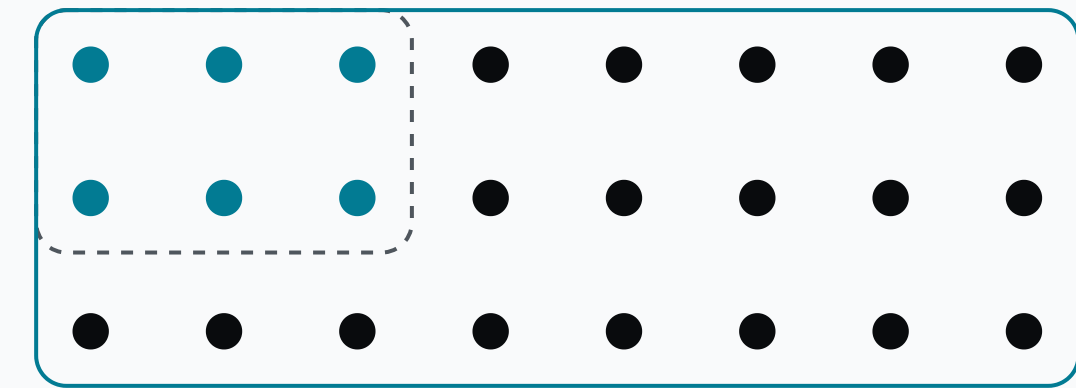
Keep every piece of work small enough to review.



E

Expand

Scale out what has proven it can be trusted.



Restrict

Scoped tasks

Only work on this module. Never touch that one.

Specialized tasks

One task to write the tests, another to build one part of the implementation.

Tests

Prevent stray modifications, and show the next agent what is in scope and what you built.

Defer

When you hit unrelated work, record it and tackle it later: update the plan, or plan a new checklist.

Contain

Plan as a checklist

Have the AI write the plan as a checklist.

One item at a time

Only the next item on the list.

Clear and continue

Tick it off, clear the context, start the next item fresh.

Rules

Set up rules in your plan for what the agent can and cannot do.

Expand

Context

It has the codebase, the plan, the decisions and the previous modifications.

Examples

The items already built are the pattern for the ones that follow.

Validated

You have reviewed and checked that part yourself.
You should be happy with this result and trust this.

Same rules

The restrictions and the checklist stay in place while it extends the work.

A bigger model is not a better result

Each model can do the others' work,
but it is best at the task it was trained for.

Opus plans

Reasons through the problem and breaks it into a checklist.

Sonnet executes

Works through that checklist, one item at a time.

Haiku writes

Fast and light for text: summaries, descriptions, documentation.

An action you should take

Do not drop your existing work. This goes on top of it.

01 _____

Build a plan

Scoped to the task, with what the AI may not touch.

02 _____

Review the plan

Read it before any code is written, and turn it into a checklist.

03 _____

Execute one by one

One item, then clear the context and take the next.

04 _____

Review the changes

Read every diff before it lands.

Any questions?

info@smartagents.be

smartagents.be

Thank you